

BCSL-022 Assembly Language Programming Lab

Complete Assignment Documentation

Course Code: BCSL-022

Course Title: Assembly Language Programming Lab

Assignment Number: BCA(II)/L-022/Assignment/2025-26

Maximum Marks: 50 (40 marks for programs + 10 marks for viva)

Session: July 2025 / January 2026

Table of Contents

1. [Question 1: Interchange Two Memory Locations](#)
 2. [Question 2: Sum and Difference](#)
 3. [Question 3: Multiply Array Elements](#)
 4. [Question 4: Print ASCII Character](#)
 5. [Question 5: Check Zero in Array](#)
 6. [Compilation and Execution Instructions](#)
 7. [8086 Register Reference](#)
 8. [Viva Voce Preparation](#)
-

Question 1: Interchange Two Memory Locations

Problem Statement

Write and run a program using 8086 assembly language that interchanges the values stored in two memory locations.

Algorithm

1. Initialize data segment
2. Display values before interchange
3. Load first value into AL register
4. Load second value into BL register
5. Store AL (old first value) into second location
6. Store BL (old second value) into first location
7. Display values after interchange

Key Concepts

- **Memory to Register Transfer:** `MOV AL, VAR1`
- **Register to Memory Transfer:** `MOV VAR1, BL`
- **Temporary Storage:** Using registers as temporary storage

Alternative Methods

1. Using Third Register (Implemented):

```
2. MOV AL, VAR1
3. MOV BL, VAR2
4. MOV VAR2, AL
5. MOV VAR1, BL
```

6. Using XOR (without third variable):

```
7. MOV AL, VAR1
8. MOV BL, VAR2
9. XOR AL, BL      ; AL = VAR1 XOR VAR2
10. XOR BL, AL      ; BL = VAR2 XOR (VAR1 XOR VAR2) = VAR1
11. XOR AL, BL      ; AL = (VAR1 XOR VAR2) XOR VAR1 = VAR2
12. MOV VAR1, BL
13. MOV VAR2, AL
```

14. Using Stack:

```
15. MOV AL, VAR1
16. PUSH AX
17. MOV AL, VAR2
18. MOV VAR1, AL
19. POP AX
20. MOV VAR2, AL
```

Sample Output

Before Interchange:
VAR1 = 25H
VAR2 = 4FH

After Interchange:
VAR1 = 4FH
VAR2 = 25H

Time Complexity

O(1) - Constant time operation

Question 2: Sum and Difference of Two Values

Problem Statement

Write and run a program using 8086 assembly language that calculates the sum and difference of two values stored in memory locations.

Algorithm

1. Initialize data segment
2. Display both numbers
3. **Calculate Sum:**
 - Load first number into AL
 - Add second number to AL
 - Store result in SUM memory location
4. **Calculate Difference:**
 - Load first number into AL
 - Subtract second number from AL
 - Store result in DIFF memory location
5. Display sum and difference

Key Instructions

- **ADD Instruction:** `ADD AL, NUM2` - Adds NUM2 to AL
- **SUB Instruction:** `SUB AL, NUM2` - Subtracts NUM2 from AL

Flags Affected

Both ADD and SUB affect the following flags:

- **CF (Carry Flag):** Set if carry/borrow occurs
- **ZF (Zero Flag):** Set if result is zero
- **SF (Sign Flag):** Set if result is negative
- **OF (Overflow Flag):** Set if signed overflow occurs
- **PF (Parity Flag):** Set if result has even parity
- **AF (Auxiliary Flag):** Set if auxiliary carry occurs

Sample Calculation

NUM1 = 45H (69 decimal)
NUM2 = 23H (35 decimal)

SUM = 45H + 23H = 68H (104 decimal)
DIFF = 45H - 23H = 22H (34 decimal)

Sample Output

First Number (NUM1) = 45H (69)
Second Number (NUM2) = 23H (35)

SUM (NUM1 + NUM2) = 68H (104)
DIFF (NUM1 - NUM2) = 22H (34)

Results stored in memory locations.

Question 3: Multiply Array Elements

Problem Statement

Write and run a program using 8086 assembly language that multiplies the elements of an array of size 4 stored in memory. The result is left in registers.

Algorithm

1. Initialize data segment
2. Display all array elements
3. Initialize result (AX = 1, DX = 0)
4. **Multiplication Loop:**
 - Load array element into BL
 - Multiply AX by BL (result in DX:AX)
 - Move to next element
 - Repeat until all elements processed
5. Display final result in DX:AX

Key Instructions

- **MUL Instruction:**
 - Format: `MUL source`
 - Operation: $AX = AL * \text{source}$ (8-bit) or $DX:AX = AX * \text{source}$ (16-bit)
 - Unsigned multiplication

Register Usage

- **SI:** Source Index register (points to array)
- **AX:** Accumulator (holds lower 16 bits of result)
- **DX:** Data register (holds upper 16 bits of result)
- **CL:** Counter for loop control

Sample Calculation

Array: [02H, 03H, 04H, 05H]

Step 1: $1 \times 02H = 02H$

Step 2: $02H \times 03H = 06H$

Step 3: $06H \times 04H = 18H$

Step 4: $18H \times 05H = 78H$ (120 decimal)

Final Result: DX:AX = 0000:0078H

Sample Output

Array Elements:

Element 1: 02H

Element 2: 03H

Element 3: 04H

Element 4: 05H

Product of all elements:

Result in DX:AX = 0000:0078H (120)

Why DX:AX?

The MUL instruction for 16-bit multiplication produces a 32-bit result:

- **DX** = Upper 16 bits
 - **AX** = Lower 16 bits
 - This allows results up to 4,294,967,295 (FFFFFFFFH)
-

Question 4: Print ASCII Character

Problem Statement

Write and run a program using 8086 assembly language that prints the ASCII character for an 8-bit value stored in BL register.

Algorithm

1. Initialize data segment
2. Store ASCII value in BL register
3. Display the hexadecimal value
4. **Print ASCII character:**
 - Move BL to DL
 - Use DOS interrupt 21H, function 02H
 - Character is displayed
5. Repeat with different test values

Key Concepts

DOS Interrupt 21H, Function 02H

```
MOV DL, character    ; Character to display
MOV AH, 02H          ; Function number
INT 21H              ; Call DOS interrupt
```

ASCII Character Ranges

Range	Characters	Description
30H-39H	'0'-'9'	Decimal digits
41H-5AH	'A'-'Z'	Uppercase letters
61H-7AH	'a'-'z'	Lowercase letters
20H	Space	Whitespace
21H-2FH	!, ", #, etc.	Special characters

Sample Output

ASCII Character Display Program

Value in BL register = 41H (65)
ASCII Character = A

Value in BL register = 5AH (90)
ASCII Character = Z

Value in BL register = 30H (48)
ASCII Character = 0

Value in BL register = 40H (64)
ASCII Character = @

Value in BL register = 24H (36)
ASCII Character = \$

Common ASCII Values

- 65 (41H) = 'A'
 - 97 (61H) = 'a'
 - 48 (30H) = '0'
 - 32 (20H) = Space
 - 64 (40H) = '@'
-

Question 5: Check Zero in Array

Problem Statement

Write and run a program using 8086 assembly language that checks if any of the values in an array of size 5 is zero. If zero is found, move the index to DL register, otherwise clear DL.

Algorithm

1. Initialize data segment
2. Display array elements
3. **Zero Check Logic:**
 - Initialize index counter (CH = 0)
 - Clear DL register
 - Loop through array:
 - Load current element
 - Compare with 00H
 - If equal, store index in DL and exit loop
 - If not equal, increment index and continue
 - If loop completes without finding zero, DL remains 0
4. Display result

Key Instructions

- **CMP Instruction:** `CMP AL, 00H` - Compares AL with 0
- **JE Instruction:** `JE LABEL` - Jump if Equal (ZF = 1)
- **JNZ Instruction:** `JNZ LABEL` - Jump if Not Zero (ZF = 0)

Flowchart Logic

```

START
  ↓
Initialize: index = 0, DL = 0
  ↓
Load array[index]
  ↓
Is element = 0? —YES—> Store index in DL → END
  ↓ NO
index++
  ↓
More elements? —YES—> (loop back)
  ↓ NO
DL = 0 (already cleared)
  ↓
END

```

Test Cases

Test Case 1: Zero at Index 2

Array: [12H, 34H, 00H, 56H, 78H]
 Result: DL = 02H

Test Case 2: Zero at Index 0

Array: [00H, 11H, 22H, 33H, 44H]
 Result: DL = 00H
 Note: DL = 0 but zero was found at index 0

Test Case 3: Zero at Index 4

Array: [15H, 25H, 35H, 45H, 00H]
 Result: DL = 04H

Test Case 4: No Zero

Array: [10H, 20H, 30H, 40H, 50H]
 Result: DL = 00H (cleared)

Sample Output

```

===== Array Zero Check Program =====
-----
Array Elements: 12H, 34H, 00H, 56H, 78H
Zero found at index: 2
DL Register value = 02H
-----
Array Elements: 00H, 11H, 22H, 33H, 44H
Zero found at index: 0

```

```
DL Register value = 00H
-----
Array Elements: 15H, 25H, 35H, 45H, 00H
Zero found at index: 4
DL Register value = 04H
-----
Array Elements: 10H, 20H, 30H, 40H, 50H
No zero found. DL cleared.
DL Register value = 00H
```

Compilation and Execution Instructions

Using MASM (Microsoft Macro Assembler)

Step 1: Assembly

```
MASM program.asm;
```

This creates `program.obj`

Step 2: Linking

```
LINK program.obj;
```

This creates `program.exe`

Step 3: Execution

```
program.exe
```

Using TASM (Turbo Assembler)

Step 1: Assembly

```
TASM program.asm
```

Step 2: Linking

```
TLINK program.obj
```

Step 3: Execution

```
program.exe
```

Using DOSBox (Recommended for Modern Systems)

1. **Install DOSBox**
2. **Mount directory:**
3. `mount c: C:\asmc:`

4. **Assemble and Link:**
5. `masm program.asm;link program.obj;`
6. **Run:**
7. `program.exe`

Using EMU8086 (Easiest Option)

1. Open EMU8086
2. Load the .asm file
3. Click "Emulate" button
4. Click "Run" to execute
5. View output in the emulator window

8086 Register Reference

General Purpose Registers (16-bit)

Register	Name	High Byte	Low Byte	Purpose
AX	Accumulator	AH	AL	Arithmetic operations
BX	Base	BH	BL	Base addressing
CX	Counter	CH	CL	Loop counter
DX	Data	DH	DL	I/O operations

Index Registers

Register	Name	Purpose
SI	Source Index	Source pointer in string operations
DI	Destination Index	Destination pointer in string operations

Pointer Registers

Register	Name	Purpose
SP	Stack Pointer	Points to top of stack
BP	Base Pointer	Base for accessing stack data

Segment Registers

Register	Name	Purpose
CS	Code Segment	Code segment address
DS	Data Segment	Data segment address
SS	Stack Segment	Stack segment address

Register	Name	Purpose
ES	Extra Segment	Extra segment address

Flags Register

Flag	Name	Bit	Purpose
CF	Carry Flag	0	Carry out of MSB
PF	Parity Flag	2	Even parity
AF	Auxiliary Flag	4	BCD arithmetic
ZF	Zero Flag	6	Result is zero
SF	Sign Flag	7	Sign of result
OF	Overflow Flag	11	Signed overflow

; BCSL-022 Assignment - Question 1

; Program: Interchange values stored in two memory locations

; Assumption: Two 8-bit values are stored at memory locations VAR1 and VAR2

.MODEL SMALL

.STACK 100H

.DATA

VAR1 DB 25H ; First memory location (Value = 25H = 37 decimal)

VAR2 DB 4FH ; Second memory location (Value = 4FH = 79 decimal)

MSG1 DB 0DH, 0AH, 'Before Interchange:', 0DH, 0AH, '\$'

MSG2 DB 0DH, 0AH, 'VAR1 = \$'

MSG3 DB 0DH, 0AH, 'VAR2 = \$'

MSG4 DB 0DH, 0AH, 0DH, 0AH, 'After Interchange:', 0DH, 0AH, '\$'

.CODE

MAIN PROC

; Initialize Data Segment

MOV AX, @DATA

MOV DS, AX

; Display "Before Interchange" message

LEA DX, MSG1

MOV AH, 09H

INT 21H

; Display VAR1 before interchange

LEA DX, MSG2

MOV AH, 09H

INT 21H

MOV AL, VAR1

CALL DISPLAY_HEX

; Display VAR2 before interchange

LEA DX, MSG3

MOV AH, 09H

INT 21H

MOV AL, VAR2

CALL DISPLAY_HEX

```

; ===== INTERCHANGE LOGIC =====
; Method: Using a temporary register
MOV AL, VAR1    ; Load VAR1 into AL
MOV BL, VAR2    ; Load VAR2 into BL
MOV VAR2, AL    ; Store AL (old VAR1) into VAR2
MOV VAR1, BL    ; Store BL (old VAR2) into VAR1
; =====

; Display "After Interchange" message
LEA DX, MSG4
MOV AH, 09H
INT 21H

; Display VAR1 after interchange
LEA DX, MSG2
MOV AH, 09H
INT 21H
MOV AL, VAR1
CALL DISPLAY_HEX

; Display VAR2 after interchange
LEA DX, MSG3
MOV AH, 09H

```

INT 21H

MOV AL, VAR2

CALL DISPLAY_HEX

; Exit program

MOV AH, 4CH

INT 21H

MAIN ENDP

; Procedure to display hexadecimal value in AL

DISPLAY_HEX PROC

PUSH AX

PUSH BX

PUSH CX

PUSH DX

MOV BL, AL ; Save original value

; Display upper nibble

MOV CL, 4

SHR AL, CL ; Shift right 4 bits

ADD AL, 30H ; Convert to ASCII

CMP AL, 39H

JLE PRINT1

ADD AL, 7 ; Adjust for A-F

PRINT1:

MOV DL, AL

MOV AH, 02H

INT 21H

; Display lower nibble

MOV AL, BL

AND AL, 0FH ; Mask upper nibble

ADD AL, 30H

CMP AL, 39H

JLE PRINT2

ADD AL, 7

PRINT2:

MOV DL, AL

MOV AH, 02H

INT 21H

; Display 'H' suffix

MOV DL, 'H'

MOV AH, 02H

INT 21H

```
POP DX
POP CX
POP BX
POP AX
RET
DISPLAY_HEX ENDP
```

```
END MAIN
```

```
; ===== ALTERNATIVE METHOD (Without
using third register) =====
```

```
; Interchange using XOR operations (commented out)
```

```
; XOR AL, BL ; AL = AL XOR BL
```

```
; XOR BL, AL ; BL = BL XOR (AL XOR BL) = AL
```

```
; XOR AL, BL ; AL = (AL XOR BL) XOR AL = BL
```

```
;
```

```
=====
=====
```

```
; ===== PROGRAM OUTPUT =====
```

```
; Before Interchange:
```

```
; VAR1 = 25H
```

```
; VAR2 = 4FH
```


;

; **After Interchange:**

; **VAR1 = 4FH**

; **VAR2 = 25H**

; **=====**